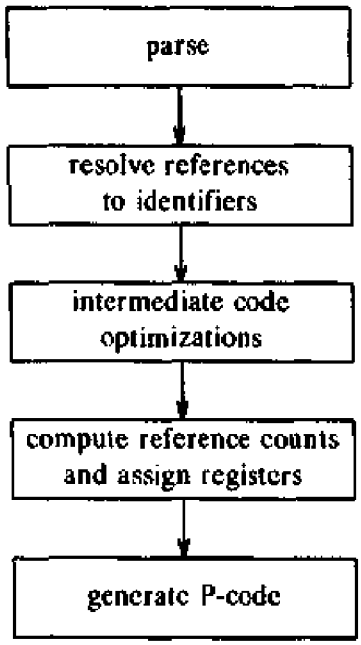

Fig. 12.6. Modula-2 compiler passes.

of Section 10.5 can be used here as well. In fact, the Modula compiler goes beyond the Bliss-11 compiler in the way it takes advantage of the syntax. Loops are identified by their syntax; i.e., the compiler looks for while- and for-constructs. Invariant expressions are detected by the fact that none of their variables are defined in the loop, and these are moved to a loop header. The only induction variables that are detected are those in the family of a for-loop index. Global common subexpressions are detected when one is in a block that dominates the block of the other, but this analysis is done an expression at a time, rather than with bit vectors.

The register allocation strategy is similarly designed to do reasonable things without being exhaustive. In particular, it considers as candidates for allocation to a register only:

1. temporaries used during the evaluation of an expression (these receive first priority),
2. values of common subexpressions,
3. indices and limit values in for-loops,
4. the address of *E* in an expression of the form `with` *E* `do`, and
5. simple variables (characters, integers, and so on) local to the current procedure.

An attempt is made to estimate the value of keeping each variable in classes (2)–(5) in a register. It is assumed that a statement is executed 10^d times if it

is nested within d loops. However, variables referenced no more than twice are not considered eligible; others are ranked in order of estimated use and assigned to a register if one is available after assigning expression temporaries and higher-ranked variables.

APPENDIX

A
Programming Project

A.1 INTRODUCTION

This appendix suggests programming exercises that can be used in a programming laboratory accompanying a compiler-design course based on this book. The exercises consist of implementing the basic components of a compiler for a subset of Pascal. The subset is minimal, but allows programs such as the recursive sorting procedure in Section 7.1 to be expressed. Being a subset of an existing language has certain utility. The meaning of programs in the subset is determined by the semantics of Pascal (Jensen and Wirth [1975]). If a Pascal compiler is available, it can be used as a check on the behavior of the compiler written as an exercise. The constructs in the subset appear in most programming languages, so corresponding exercises can be formulated using a different language if a Pascal compiler is not available.

A.2 PROGRAM STRUCTURE

A program consists of a sequence of global data declarations, a sequence of procedure and function declarations, and a single compound statement that is the "main program." Global data is to be allocated static storage. Data local to procedures and functions is allocated storage on a stack. Recursion is permitted, and parameters are passed by reference. The procedures `read` and `write` are assumed supplied by the compiler.

Fig. A.1 gives an example program. The name of the program is `example`, and `input` and `output` are the names of the files used by `read` and `write`, respectively.

A.3 SYNTAX OF A PASCAL SUBSET

Listed below is an LALR(1) grammar for a subset of Pascal. The grammar can be modified for recursive-descent parsing by eliminating left recursion as described in Sections 2.4 and 4.3. An operator-precedence parser can be

```
program example(input, output);
var x, y: integer;
function gcd(a, b: integer): integer;
begin
    if b = 0 then gcd := a
    else gcd := gcd(b, a mod b)
end;

begin
    read(x, y);
    write(gcd(x, y))
end.
```

Fig. A.1. Example program.

constructed for expressions by substituting out for **relop**, **addop**, and **mulop**, and eliminating ϵ-productions.

The addition of the production

statement → **if** *expression* **then** *statement*

introduces the "dangling-else" ambiguity, which can be eliminated as discussed in Section 4.3 (see also Example 4.19 if predictive parsing is used).

There is no syntactic distinction between a simple variable and the call of a function without parameters. Both are generated by the production

factor → **id**

Thus, the assignment `a := b` sets `a` to the value returned by the function `b`, if `b` has been declared to be a function.

program →
    program id (*identifier_list*) ;
    declarations
    subprogram_declarations
    compound_statement
    .

identifier_list →
    id
    | *identifier_list* , **id**

declarations →
    declarations **var** *identifier_list* : *type* ;
    | ϵ

type →
    standard_type
    | **array** [**num** . . **num**] **of** *standard_type*

standard_type →
    integer
    | **real**

subprogram_declarations →
    subprogram_declarations subprogram_declaration ;
    | ϵ

subprogram_declaration →
    subprogram_head declarations compound_statement

subprogram_head →
    function id *arguments* : *standard_type* ;
    | **procedure id** *arguments* ;

arguments →
    (*parameter_list*)
    | ϵ

parameter_list →
    identifier_list : *type*
    | *parameter_list* ; *identifier_list* : *type*

compound_statement →
    begin
    optional_statements
    end

optional_statements →
    statement_list
    | ϵ

statement_list →
    statement
    | *statement_list* ; *statement*

statement →
    variable **assignop** *expression*
    | *procedure_statement*
    | *compound_statement*
    | **if** *expression* **then** *statement* **else** *statement*
    | **while** *expression* **do** *statement*

variable →
    id
    | **id** [*expression*]

procedure_statement →
    id
    | **id** (*expression_list*)

expression_list →
    expression
    | *expression_list* , *expression*

expression →
    simple_expression
    | *simple_expression* **relop** *simple_expression*

simple_expression →
    term
    | *sign term*
    | *simple_expression* **addop** *term*

term →
    factor
    | *term* **mulop** *factor*

factor →
    id
    | **id** (*expression_list*)
    | **num**
    | (*expression*)
    | **not** *factor*

sign →
    + | -

A.4 LEXICAL CONVENTIONS

The notation for the specifying tokens is from Section 3.3.

1. Comments are surrounded by { and }. They may not contain a {. Comments may appear after any token.

2. Blanks between tokens are optional, with the exception that keywords must be surrounded by blanks, newlines, the beginning of the program, or the final dot.

3. Token **id** for identifiers matches a letter followed by letters or digits:

    letter → [a-zA-Z]
    digit → [0-9]
    id → **letter** (**letter** | **digit**)*

    The implementer may wish to put a limit on identifier length.

4. Token **num** matches unsigned integers (see Example 3.5):

    digits → **digit digit***
    optional_fraction → . **digits** | ϵ
    optional_exponent → (E (+ | - | ϵ) **digits**) | ϵ
    num → **digits optional_fraction optional_exponent**

5. Keywords are reserved and appear in boldface in the grammar.

6. The relation operators (**relop**'s) are: =, <>, <, <=, >=, and >. Note that <> denotes $\neq$.

7. The **addop**'s are +, -, and or.

8. The **mulop**'s are *, /, div, mod, and and.

9. The lexeme for token **assignop** is :=.

A.5 SUGGESTED EXERCISES

A programming exercise suitable for a one-term course is to write an interpreter for the language defined above, or for a similar subset of another high-level language. The project involves translating the source program into an intermediate representation such as quadruples or stack machine code and then interpreting the intermediate representation. We shall propose an order for the construction of the modules. The order is different from the order in which the modules are executed in the compiler because it is convenient to have a working interpreter to debug the other compiler components.

1. *Design a symbol-table mechanism.* Decide on the symbol-table organization. Allow for information to be collected about names, but leave the symbol-table record structure flexible at this time. Write routines to:

i) Search the symbol table for a given name, create a new entry for that name if none is present, and in either case return a pointer to the record for that name.

ii) Delete from the symbol table all names local to a given procedure.

2. *Write an interpreter for quadruples.* The exact set of quadruples may be left open at this time but they should include the arithmetic and conditional jump statements corresponding to the set of operators in the language. Also include logical operations if conditions are evaluated arithmetically rather than by position in the program. In addition, expect to need "quadruples" for integer-to-real conversion, for marking the beginning and end of procedures, and for parameter passing and procedure calls.

It is also necessary at this time to design the calling sequence and runtime organization for the programs being interpreted. The simple stack organization discussed in Section 7.3 is suitable for the example language, because no nested declarations of procedures are permitted in the language; that is, variables are either global (declared at the level of the entire program) or local to a simple procedure.

For simplicity, another high-level language may be used in place of the interpreter. Each quadruple can be a statement of a high-level language such as C, or even Pascal. The output of the compiler is then a sequence of C statements that can be compiled on an existing C compiler. This approach enables the implementer to concentrate on the run-time organization.

3. *Write the lexical analyzer.* Select internal codes for the tokens. Decide how constants will be represented in the compiler. Count lines for later use by an error-message handler. Produce a listing of the source program if desired. Write a program to enter the reserved words into the symbol table. Design your lexical analyzer to be a subroutine called by the parser, returning a pair (token, attribute value). At present, errors detected by your lexical analyzer may be handled by calling an error-printing routine and halting.

4. *Write the semantic actions.* Write semantic routines to generate the quadruples. The grammar will need to be modified in places to make the translation easier. Consult Sections 5.5 and 5.6 for examples of how to modify the grammar usefully. Do semantic analysis at this time, converting integers to reals when necessary.

5. *Write the parser.* If an LALR parser generator is available, this will simplify the task considerably. If a parser generator handling ambiguous grammars, like Yacc, is available, then nonterminals denoting expressions can be combined. Moreover, the "dangling-else" ambiguity can be resolved by shifting whenever a shift/reduce conflict occurs.

6. *Write the error-handling routines.* Be prepared to recover from lexical and syntactic errors. Print error diagnostics for lexical, syntactic, and semantic errors.

7. *Evaluation.* The program in Fig. FIB.1 can serve as a simple test routine. Another test program can be based on the Pascal program in Fig. 7.1. The code for function `partition` in the figure corresponds to the marked fragment in the C program of Fig. 10.2. Run your compiler through a profiler, if one is available. Determine the routines in which most of the time is being spent. What modules would have to be modified in order to increase the speed of your compiler?

A.6 EVOLUTION OF THE INTERPRETER

An alternative approach to constructing an interpreter for the language is to start by implementing a desk calculator, that is, an interpreter for expressions. Gradually add constructs to the language until an interpreter for the entire language is obtained. A similar approach is taken in Kernighan and Pike [1984]. A proposed order for adding constructs is:

1. *Translate expressions into postfix notation.* Using either recursive-descent parsing, as in Chapter 2, or a parser generator, familiarize yourself with the programming environment by writing a translator from simple arithmetic expressions into postfix notation.

2. *Add a lexical analyzer.* Allow for keywords, identifiers, and numbers to appear in the translator constructed above. Retarget the translator to produce either code for a stack machine or quadruples.

3. *Write an interpreter for the intermediate representation.* As discussed in

Section A.5, a high-level language may be used in place of the interpreter. For the moment, the interpreter need only support arithmetic operations, assignments, and input-output. Extend the language by allowing global variable declarations, assignments, and calls of procedures `read` and `write`. These constructs allow the interpreter to be tested.

4. *Add statements.* A program in the language now consists of a main program without subprogram declarations. Test both the translator and the interpreter.

5. *Add procedures and functions.* The symbol table must now allow the scopes of identifiers to be limited to procedure bodies. Design a calling sequence. Again, the simple stack organization of Section 7.3 is adequate. Extend the interpreter to support the calling sequence.

A.7 EXTENSIONS

There are a number of features that can be added to the language without greatly increasing the complexity of compilation. Among these are:

1. multidimensional arrays
2. for- and case-statements
3. block structure
4. record structures

If time permits, add one or more of these extensions to your compiler.

Bibliography

ABEL, N. E. AND J. R. BELL [1972]. "Global optimization in compilers," *Proc. First USA-Japan Computer Conf.*, AFIPS Press, Montvale, N. J.

ABELSON, H. AND G. J. SUSSMAN [1985]. *Structure and Interpretation of Computer Programs*, MIT Press, Cambridge, Mass.

ADRION, W. R., M. A. BRANSTAD, AND J. C. CHERNIAVSKY [1982]. "Validation, verification, and testing of computer software," *Computing Surveys* **14:2**, 159-192.

AHO, A. V. [1980]. "Pattern matching in strings," in Book [1980], pp. 325-347.

AHO, A. V. AND M. J. CORASICK [1975]. "Efficient string matching: an aid to bibliographic search," *Comm. ACM* **18:6**, 333-340.

AHO, A. V. AND M. GANAPATHI [1985]. "Efficient tree pattern matching: an aid to code generation," *Twelfth Annual ACM Symposium on Principles of Programming Languages*, 334-340.

AHO, A. V., J. E. HOPCROFT, AND J. D. ULLMAN [1974]. *The Design and Analysis of Computer Algorithms*, Addison-Wesley, Reading, Mass.

AHO, A. V., J. E. HOPCROFT, AND J. D. ULLMAN [1983]. *Data Structures and Algorithms*, Addison-Wesley, Reading, Mass.

AHO, A. V. AND S. C. JOHNSON [1974]. "LR parsing," *Computing Surveys* **6:2**, 99-124.

AHO, A. V. AND S. C. JOHNSON [1976]. "Optimal code generation for expression trees," *J. ACM* **23:3**, 488-501.

AHO, A. V., S. C. JOHNSON, AND J. D. ULLMAN [1975]. "Deterministic parsing of ambiguous grammars," *Comm. ACM* **18:8**, 441-452.

AHO, A. V., S. C. JOHNSON, AND J. D. ULLMAN [1977a]. "Code generation for expressions with common subexpressions," *J. ACM* **24:1**, 146-160.

AHO, A. V., S. C. JOHNSON, AND J. D. ULLMAN [1977b]. "Code generation for machines with multiregister operations," *Fourth ACM Symposium on Principles of Programming Languages*, 21-28.

AHO, A. V., B. W. KERNIGHAN, AND P. J. WEINBERGER [1979]. "Awk – a

pattern scanning and processing language," *Software—Practice and Experience* **9:4**, 267-280.

AHO, A. V. AND T. G. PETERSON [1972]. "A minimum distance error-correcting parser for context-free languages," *SIAM J. Computing* **1:4**, 305-312.

AHO, A. V. AND R. SETHI [1977]. "How hard is compiler code generation?" Lecture Notes in Computer Science **52**, Springer-Verlag, Berlin, 1-15.

AHO, A. V. AND J. D. ULLMAN [1972a]. "Optimization of straight line code," *SIAM J. Computing* **1:1**, 1-19.

AHO, A. V. AND J. D. ULLMAN [1972b]. *The Theory of Parsing, Translation and Compiling*, Vol. I: *Parsing*, Prentice-Hall, Englewood Cliffs, N. J.

AHO, A. V. AND J. D. ULLMAN [1973a]. *The Theory of Parsing, Translation and Compiling*, Vol. II: *Compiling*, Prentice-Hall, Englewood Cliffs, N. J.

AHO, A. V. AND J. D. ULLMAN [1973b]. "A technique for speeding up LR(k) parsers," *SIAM J. Computing* **2:2**, 106-127.

AHO, A. V. AND J. D. ULLMAN [1977]. *Principles of Compiler Design*, Addison-Wesley, Reading, Mass.

AIGRAIN, P., S. L. GRAHAM, R. R. HENRY, M. K. MCKUSICK, AND E. PELEGRI-LLOPART [1984]. "Experience with a Graham-Glanville style code generator," *ACM SIGPLAN Notices* **19:6**, 13-24.

ALLEN, F. E. [1969]. "Program optimization," *Annual Review in Automatic Programming* **5**, 239-307.

ALLEN, F. E. [1970]. "Control flow analysis," *ACM SIGPLAN Notices* **5:7**, 1-19.

ALLEN, F. E. [1974]. "Interprocedural data flow analysis," *Information Processing 74*, North-Holland, Amsterdam, 398-402.

ALLEN, F. E. [1975]. "Bibliography on program optimization," RC-5767, IBM T. J. Watson Research Center, Yorktown Heights, N. Y.

ALLEN, F. E., J. L. CARTER, J. FABRI, J. FERRANTE, W. H. HARRISON, P. G. LOEWNER, AND L. H. TREVILLYAN [1980]. "The experimental compiling system," *IBM. J. Research and Development* **24:6**, 695-715.

ALLEN, F. E. AND J. COCKE [1972]. "A catalogue of optimizing transformations," in Rustin [1972], pp. 1-30.

ALLEN, F. E. AND J. COCKE [1976]. "A program data flow analysis procedure," *Comm. ACM* **19:3**, 137-147.

ALLEN, F. E., J. COCKE, AND K. KENNEDY [1981]. "Reduction of operator strength," in Muchnick and Jones [1981], pp. 79-101.

AMMANN, U. [1977]. "On code generation in a Pascal compiler," *Software—Practice and Experience* **7:3**, 391-423.

AMMANN, U. [1981]. "The Zurich implementation," in Barron [1981], pp. 63-82.

ANDERSON, J. P. [1964]. "A note on some compiling algorithms," *Comm. ACM* **7:3**, 149-150.

ANDERSON, T., J. EVE, AND J. J. HORNING [1973]. "Efficient LR(1) parsers," *Acta Informatica* **2:1**, 12-39.

ANKLAM, P., D. CUTLER, R. HEINEN, JR., AND M. D. MACLAREN [1982]. *Engineering a Compiler*, Digital Press, Bedford, Mass.

ARDEN, B. W., B. A. GALLER, AND R. M. GRAHAM [1961]. "An algorithm for equivalence declarations," *Comm. ACM* **4:7**, 310-314.

AUSLANDER, M. A. AND M. E. HOPKINS [1982]. "An overview of the PL.8 compiler," *ACM SIGPLAN Notices* **17:6**, 22-31.

BACKHOUSE, R. C. [1976]. "An alternative approach to the improvement of LR parsers," *Acta Informatica* **6:3**, 277-296.

BACKHOUSE, R. C. [1984]. "Global data flow analysis problems arising in locally least-cost error recovery," *TOPLAS* **6:2**, 192-214.

BACKUS, J. W. [1981]. "Transcript of presentation on the history of Fortran I, II, and III," in Wexelblat [1981], pp. 45-66.

BACKUS, J. W., R. J. BEEBER, S. BEST, R. GOLDBERG, L. M. HAIBT, H. L. HERRICK, R. A. NELSON, D. SAYRE, P. B. SHERIDAN, H. STERN, I. ZILLER, R. A. HUGHES, AND R. NUTT [1957]. "The Fortran automatic coding system," *Western Joint Computer Conference*, 188-198. Reprinted in Rosen [1967], pp. 29-47.

BAKER, B. S. [1977]. "An algorithm for structuring programs," *J. ACM* **24:1**, 98-120.

BAKER, T. P. [1982]. "A one-pass algorithm for overload resolution in Ada," *TOPLAS* **4:4**, 601-614.

BANNING, J. P. [1979]. "An efficient way to find the side effects of procedure calls and aliases of variables," *Sixth Annual ACM Symposium on Principles of Programming Languages*, 29-41.

BARRON, D. W. [1981]. *Pascal – The Language and its Implementation*, Wiley, Chichester.

BARTH, J. M. [1978]. "A practical interprocedural data flow analysis algorithm," *Comm. ACM* **21:9**, 724-736.

BATSON, A. [1965]. "The organization of symbol tables," *Comm. ACM* **8:2**, 111-112.

BAUER, A. M. AND H. J. SAAL [1974]. "Does APL really need run-time checking?" *Software—Practice and Experience* **4:2**, 129-138.

BAUER, F. L. [1976]. "Historical remarks on compiler construction," in Bauer and Eickel [1976], pp. 603-621. Addendum by A. P. Ershov, pp. 622-626.

BAUER, F. L. AND J. EICKEL [1976]. *Compiler Construction: An Advanced Course*, 2nd Ed., Lecture Notes in Computer Science **21**, Springer-Verlag, Berlin.

BAUER, F. L. AND H. WOSSNER [1972]. "The 'Plankankül' of Konrad Zuse: A forerunner of today's programming languages," *Comm. ACM* **15:7**, 678-685.

BEATTY, J. C. [1972]. "An axiomatic approach to code optimization for expressions," *J. ACM* **19:4**, 714-724. Errata **20** (1973), p. 180 and 538.

BEATTY, J. C. [1974]. "Register assignment algorithm for generation of highly optimized object code," *IBM J. Research and Development* **5:2**, 20-39.

BELADY, L. A. [1966]. "A study of replacement algorithms for a virtual storage computer," *IBM Systems J.* **5:2**, 78-101.

BENTLEY, J. L. [1982]. *Writing Efficient Programs*, Prentice-Hall, Englewood Cliffs, N. J.

BENTLEY, J. L., W. S. CLEVELAND, AND R. SETHI [1985]. "Empirical analysis of hash functions," manuscript, AT&T Bell Laboratories, Murray Hill, N. J.

BIRMAN, A. AND J. D. ULLMAN [1973]. "Parsing algorithms with backtrack," *Information and Control* **23:1**, 1-34.

BOCHMANN, G. V. [1976]. "Semantic evaluation from left to right," *Comm. ACM* **19:2**, 55-62.

BOCHMANN, G. V. AND P. WARD [1978]. "Compiler writing system for attribute grammars," *Computer J.* **21:2**, 144-148.

BOOK, R. V. [1980]. *Formal Language Theory*, Academic Press, New York.

BOYER, R. S. AND J S. MOORE [1977]. "A fast string searching algorithm," *Comm. ACM* **20:10**, 262-272.

BRANQUART, P., J.-P. CARDINAEL, J. LEWI, J.-P. DELESCAILLE, AND M. VANBEGIN [1976]. *An Optimized Translation Process and its Application to Algol 68*, Lecture Notes in Computer Science, **38**, Springer-Verlag, Berlin.

BRATMAN, H. [1961]. "An alternate form of the 'Uncol diagram'," *Comm. ACM* **4:3**, 142.

BROOKER, R. A. AND D. MORRIS [1962]. "A general translation program for phrase structure languages," *J. ACM* **9:1**, 1-10.

BROOKS, F. P., JR. [1975]. *The Mythical Man-Month*, Addison-Wesley, Reading, Mass.

BROSGOL, B. M. [1974]. *Deterministic Translation Grammars*, Ph. D. Thesis, TR 3-74, Harvard Univ., Cambridge, Mass.

BRUNO, J. AND T. LASSAGNE [1975]. "The generation of optimal code for stack machines," *J. ACM* **22:3**, 382-396.

BRUNO, J. AND R. SETHI [1976]. "Code generation for a one-register machine," *J. ACM* **23:3**, 502-510.

BURSTALL, R. M., D. B. MACQUEEN, AND D. T. SANNELLA [1980]. "Hope: an experimental applicative language," *Lisp Conference*, P.O. Box 487, Redwood Estates, Calif. 95044, 136-143.

BUSAM, V. A. AND D. E. ENGLUND [1969]. "Optimization of expressions in Fortran," *Comm. ACM* **12:12**, 666-674.

CARDELLI, L. [1984]. "Basic polymorphic typechecking," Computing Science Technical Report 112, AT&T Bell Laboratories, Murray Hill, N. J.

CARTER, L. R. [1982]. *An Analysis of Pascal Programs*, UMI Research Press, Ann Arbor, Michigan.

CARTWRIGHT, R. [1985]. "Types as intervals," *Twelfth Annual ACM Symposium on Principles of Programming Languages*, 22-36.

CATTELL, R. G. G. [1980]. "Automatic derivation of code generators from machine descriptions," *TOPLAS* **2:2**, 173-190.

CHAITIN, G. J. [1982]. "Register allocation and spilling via graph coloring," *ACM SIGPLAN Notices* **17:6**, 201-207.

CHAITIN, G. J., M. A. AUSLANDER, A. K. CHANDRA, J. COCKE, M. E. HOPKINS, AND P. W. MARKSTEIN [1981]. "Register allocation via coloring," *Computer Languages* **6**, 47-57.

CHERNIAVSKY, J. C., P. B. HENDERSON, AND J. KEOHANE [1976]. "On the equivalence of URE flow graphs and reducible flow graphs," *Proc. 1976 Conference on Information Sciences and Systems*, Johns Hopkins Univ., 423-429.

CHERRY, L. L. [1982]. "Writing tools," *IEEE Trans. on Communications* **COM-30:1**, 100-104.

CHOMSKY, N. [1956]. "Three models for the description of language," *IRE*

Trans. on Information Theory **IT-2:3**, 113-124.

CHOW, F. [1983]. *A Portable Machine-Independent Global Optimizer*, Ph. D. Thesis, Computer System Lab., Stanford Univ., Stanford, Calif.

CHOW, F. AND J. L. HENNESSY [1984]. "Register allocation by priority-based coloring," *ACM SIGPLAN Notices* **19:6**, 222-232.

CHURCH, A. [1941]. *The Calculi of Lambda Conversion*, Annals of Math. Studies, No. 6, Princeton University Press, Princeton, N. J.

CHURCH, A. [1956]. *Introduction to Mathematical Logic*, Vol. 1, Princeton University Press, Princeton, N. J.

CIESINGER, J. [1979]. "A bibliography of error handling," *ACM SIGPLAN Notices* **14:1**, 16-26.

COCKE, J. [1970]. "Global common subexpression elimination," *ACM SIGPLAN Notices* **5:7**, 20-24.

COCKE, J. AND K. KENNEDY [1976]. "Profitability computations on program flow graphs," *Computers and Mathematics with Applications* **2:2**, 145-159.

COCKE, J. AND K. KENNEDY [1977]. "An algorithm for reduction of operator strength," *Comm. ACM* **20:11**, 850-856.

COCKE, J. AND J. MARKSTEIN [1980]. "Measurement of code improvement algorithms," *Information Processing 80*, 221-228.

COCKE, J. AND J. MILLER [1969]. "Some analysis techniques for optimizing computer programs," *Proc. 2nd Hawaii Intl. Conf. on Systems Sciences*, 143-146.

COCKE, J. AND J. T. SCHWARTZ [1970]. *Programming Languages and Their Compilers: Preliminary Notes, Second Revised Version*, Courant Institute of Mathematical Sciences, New York.

COFFMAN, E. G., JR. AND R. SETHI [1983]. "Instruction sets for evaluating arithmetic expressions," *J. ACM* **30:3**, 457-478.

COHEN, R. AND E. HARRY [1979]. "Automatic generation of near-optimal linear-time translators for non-circular attribute grammars," *Sixth ACM Symposium on Principles of Programming Languages*, 121-134.

CONWAY, M. E. [1963]. "Design of a separable transition diagram compiler," *Comm. ACM* **6:7**, 396-408.

CONWAY, R. W. AND W. L. MAXWELL [1963]. "CORC – the Cornell computing language," *Comm. ACM* **6:6**, 317-321.

CONWAY, R. W. AND T. R. WILCOX [1973]. "Design and implementation of a diagnostic compiler for PL/I," *Comm. ACM* **16:3**, 169-179.

CORMACK, G. V. [1981]. "An algorithm for the selection of overloaded

functions in Ada," *ACM SIGPLAN Notices* **16:2** (February) 48-52.

CORMACK, G. V., R. N. S. HORSPOOL, AND M. KAISERSWERTH [1985]. "Practical perfect hashing," *Computer J.* **28:1**, 54-58.

COURCELLE, B. [1984]. "Attribute grammars: definitions, analysis of dependencies, proof methods," in Lorho [1984], pp. 81-102.

COUSOT, P. [1981]. "Semantic foundations of program analysis," in Muchnick and Jones [1981], pp. 303-342.

COUSOT, P. AND R. COUSOT [1977]. "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints," *Fourth ACM Symposium on Principles of Programming Languages*, 238-252.

CURRY, H. B. AND R. FEYS [1958]. *Combinatory Logic*, Vol. 1, North-Holland, Amsterdam.

DATE, C. J. [1986]. *An Introduction to Database Systems*, 4th Ed., Addison-Wesley, Reading, Mass.

DAVIDSON, J. W. AND C. W. FRASER [1980]. "The design and application of a retargetable peephole optimizer," *TOPLAS* **2:2**, 191-202. Errata **3:1** (1981) 110.

DAVIDSON, J. W. AND C. W. FRASER [1984a]. "Automatic generation of peephole optimizations," *ACM SIGPLAN Notices* **19:6**, 111-116.

DAVIDSON, J. W. AND C. W. FRASER [1984b]. "Code selection through object code optimization," *TOPLAS* **6:4**, 505-526.

DEREMER, F. [1969]. *Practical Translators for LR(k) Languages*, Ph. D. Thesis, M.I.T., Cambridge, Mass.

DEREMER, F. [1971]. "Simple LR(k) grammars," *Comm. ACM* **14:7**, 453-460.

DEREMER, F. AND T. PENNELLO [1982]. "Efficient computation of LALR(1) look-ahead sets," *TOPLAS* **4:4**, 615-649.

DEMERS, A. J. [1975]. "Elimination of single productions and merging of nonterminal symbols in LR(1) grammars," *J. Computer Languages* **1:2**, 105-119.

DENCKER, P., K. DURRE, AND J. HEUFT [1984]. "Optimization of parser tables for portable compilers," *TOPLAS* **6:4**, 546-572.

DERANSART, P., M. JOURDAN, AND B. LORHO [1984]. "Speeding up circularity tests for attribute grammars," *Acta Informatica* **21**, 375-391.

DESPEYROUX, T. [1984]. "Executable specifications of static semantics," in Kahn, MacQueen, and Plotkin [1984], pp. 215-233.

DIJKSTRA, E. W. [1960]. "Recursive programming," *Numerische Math.* **2**,

312-318. Reprinted in Rosen [1967], pp. 221-228.

DIJKSTRA, E. W. [1963]. "An Algol 60 translator for the X1," *Annual Review in Automatic Programming* **3**, Pergamon Press, New York, 329-345.

DITZEL, D. AND H. R. MCLELLAN [1982]. "Register allocation for free: the C machine stack cache," *Proc. ACM Symp. on Architectural Support for Programming Languages and Operating Systems*, 48-56.

DOWNEY, P. J. AND R. SETHI [1978]. "Assignment commands with array references," *J. ACM* **25:4**, 652-666.

DOWNEY, P. J., R. SETHI, AND R. E. TARJAN [1980]. "Variations on the common subexpression problem," *J. ACM* **27:4**, 758-771.

EARLEY, J. [1970]. "An efficient context-free parsing algorithm," *Comm. ACM* **13:2**, 94-102.

EARLEY, J. [1975a]. "Ambiguity and precedence in syntax description," *Acta Informatica* **4:2**, 183-192.

EARLEY, J. [1975b]. "High level iterators and a method of data structure choice," *J. Computer Languages* **1:4**, 321-342.

ELSHOFF, J. L. [1976]. "An analysis of some commercial PL/I programs," *IEEE Trans. Software Engineering* **SE2:2**, 113-120.

ENGELFRIET, J. [1984]. "Attribute evaluation methods," in Lorho [1984], pp. 103-138.

ERSHOV, A. P. [1958]. "On programming of arithmetic operations," *Comm. ACM* **1:8** (August) 3-6. Figures 1-3 appear in **1:9** (September 1958), p. 16.

ERSHOV, A. P. [1966]. "Alpha – an automatic programming system of high efficiency," *J. ACM* **13:1**, 17-24.

ERSHOV, A. P. [1971]. *The Alpha Automatic Programming System*, Academic Press, New York.

ERSHOV, A. P. AND C. H. A. KOSTER [1977]. *Methods of Algorithmic Language Implementation*, Lecture Notes in Computer Science **47**, Springer-Verlag, Berlin.

FANG, I. [1972]. "FOLDS, a declarative formal language definition system," STAN-CS-72-329, Stanford Univ.

FARROW, R. [1984]. "Generating a production compiler from an attribute grammar," *IEEE Software* **1** (October) 77-93.

FARROW, R. AND D. YELLIN [1985]. "A comparison of storage optimizations in automatically-generated compilers," manuscript, Columbia Univ.

FELDMAN, S. I. [1979a]. "Make – a program for maintaining computer

programs," *Software—Practice and Experience* **9:4**, 255-265.

FELDMAN, S. I. [1979b]. "Implementation of a portable Fortran 77 compiler using modern tools," *ACM SIGPLAN Notices* **14:8**, 98-106.

FISCHER, M. J. [1972]. "Efficiency of equivalence algorithms," in Miller and Thatcher [1972], pp. 153-168.

FLECK, A. C. [1976]. "The impossibility of content exchange through the by-name parameter transmission technique," *ACM SIGPLAN Notices* **11:11** (November) 38-41.

FLOYD, R. W. [1961]. "An algorithm for coding efficient arithmetic expressions," *Comm. ACM* **4:1**, 42-51.

FLOYD, R. W. [1963]. "Syntactic analysis and operator precedence," *J. ACM* **10:3**, 316-333.

FLOYD, R. W. [1964]. "Bounded context syntactic analysis," *Comm. ACM* **7:2**, 62-67.

FONG, A. C. [1979]. "Automatic improvement of programs in very high level languages," *Sixth Annual ACM Symposium on Principles of Programming Languages*, 21-28.

FONG, A. C. AND J. D. ULLMAN [1976]. "Induction variables in very high-level languages," *Third Annual ACM Symposium on Principles of Programming Languages*, 104-112.

FOSDICK, L. D. AND L. J. OSTERWEIL [1976]. "Data flow analysis in software reliability," *Computing Surveys* **8:3**, 305-330.

FOSTER, J. M. [1968]. "A syntax improving program," *Computer J.* **11:1**, 31-34.

FRASER, C. W. [1977]. *Automatic Generation of Code Generators*, Ph. D. Thesis, Yale Univ., New Haven, Conn.

FRASER, C. W. [1979]. "A compact, machine-independent peephole optimizer," *Sixth Annual ACM Symposium on Principles of Programming Languages*, 1-6.

FRASER, C. W. AND D. R. HANSON [1982]. "A machine-independent linker," *Software—Practice and Experience* **12**, 351-366.

FREDMAN, M. L., J. KOMLOS, AND E. SZEMEREDI [1984]. "Storing a sparse table with $O(1)$ worst case access time," *J. ACM* **31:3**, 538-544.

FREGE, G. [1879]. "Begriffsschrift, a formula language, modeled upon that of arithmetic, for pure thought," in Heijenoort [1967], 1-82.

FREIBURGHOUSE, R. A. [1969]. "The Multics PL/1 compiler," *AFIPS Fall Joint Computer Conference* **35**, 187-208.

FREIBURGHOUSE, R. A. [1974]. "Register allocation via usage counts," *Comm. ACM* **17:11**, 638-642.

FREUDENBERGER, S. M. [1984]. "On the use of global optimization algorithms for the detection of semantic programming errors," NSO-24, New York Univ.

FREUDENBERGER, S. M., J. T. SCHWARTZ, AND M. SHARIR [1983]. "Experience with the SETL optimizer," *TOPLAS* **5:1**, 26-45.

GAJEWSKA, H. [1975]. "Some statistics on the usage of the C language," AT&T Bell Laboratories, Murray Hill, N. J.

GALLER, B. A. AND M. J. FISCHER [1964]. "An improved equivalence algorithm," *Comm. ACM* **7:5**, 301-303.

GANAPATHI, M. [1980]. *Retargetable Code Generation and Optimization using Attribute Grammars*, Ph. D. Thesis, Univ. of Wisconsin, Madison, Wis.

GANAPATHI, M. AND C. N. FISCHER [1982]. "Description-driven code generation using attribute grammars," *Ninth ACM Symposium on Principles of Programming Languages*, 108-119.

GANAPATHI, M., C. N. FISCHER, AND J. L. HENNESSY [1982]. "Retargetable compiler code generation," *Computing Surveys* **14:4**, 573-592.

GANNON, J. D. AND J. J. HORNING [1975]. "Language design for programming reliability," *IEEE Trans. Software Engineering* **SE-1:2**, 179-191.

GANZINGER, H., R. GIEGERICH, U. MONCKE, AND R. WILHELM [1982]. "A truly generative semantics-directed compiler generator," *ACM SIGPLAN Notices* **17:6** (June) 172-184.

GANZINGER, H. AND K. RIPKEN [1980]. "Operator identification in Ada," *ACM SIGPLAN Notices* **15:2** (February) 30-42.

GAREY, M. R. AND D. S. JOHNSON [1979]. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco.

GEAR, C. W. [1965]. "High speed compilation of efficient object code," *Comm. ACM* **8:8**, 483-488.

GESCHKE, C. M. [1972]. *Global Program Optimizations*, Ph. D. Thesis, Dept. of Computer Science, Carnegie-Mellon Univ.

GIEGERICH, R. [1983]. "A formal framework for the derivation of machine-specific optimizers," *TOPLAS* **5:3**, 422-448.

GIEGERICH, R. AND R. WILHELM [1978]. "Counter-one-pass features in one-pass compilation: a formalization using attribute grammars," *Information Processing Letters* **7:6**, 279-284.

GLANVILLE, R. S. [1977]. *A Machine Independent Algorithm for Code*

Generation and its Use in Retargetable Compilers, Ph. D. Thesis, Univ. of California, Berkeley.

GLANVILLE, R. S. AND S. L. GRAHAM [1978]. "A new method for compiler code generation," *Fifth ACM Symposium on Principles of Programming Languages*, 231-240.

GRAHAM, R. M. [1964]. "Bounded context translation," *AFIPS Spring Joint Computer Conference* **40**, 205-217. Reprinted in Rosen [1967], pp. 184-205.

GRAHAM, S. L. [1980]. "Table-driven code generation," *Computer* **13:8**, 25-34.

GRAHAM, S. L. [1984]. "Code generation and optimization," in Lorho [1984], pp. 251-288.

GRAHAM, S. L., C. B. HALEY, AND W. N. JOY [1979]. "Practical LR error recovery," *ACM SIGPLAN Notices* **14:8**, 168-175.

GRAHAM, S. L., M. A. HARRISON, AND W. L. RUZZO [1980]. "An improved context-free recognizer," *TOPLAS* **2:3**, 415-462.

GRAHAM, S. L. AND S. P. RHODES [1975]. "Practical syntactic error recovery," *Comm. ACM* **18:11**, 639-650.

GRAHAM, S. L. AND M. WEGMAN [1976]. "A fast and usually linear algorithm for global data flow analysis," *J. ACM* **23:1**, 172-202.

GRAU, A. A., U. HILL, AND H. LANGMAACK [1967]. *Translation of Algol 60*, Springer-Verlag, New York.

HANSON, D. R. [1981]. "Is block structure necessary?" *Software—Practice and Experience* **11**, 853-866.

HARRISON, M. C. [1971]. "Implementation of the substring test by hashing," *Comm. ACM* **14:12**, 777-779.

HARRISON, W. [1975]. "A class of register allocation algorithms," RC-5342, IBM T. J. Watson Research Center, Yorktown Heights, N. Y.

HARRISON, W. [1977]. "Compiler analysis of the value ranges for variables," *IEEE Trans. Software Engineering* **3:3**.

HECHT, M. S. [1977]. *Flow Analysis of Computer Programs*, North-Holland, New York.

HECHT, M. S. AND J. B. SHAFFER [1975]. "Ideas on the design of a 'quad improver' for SIMPL-T, part I: overview and intersegment analysis," Dept. of Computer Science, Univ. of Maryland, College Park, Md.

HECHT, M. S. AND J. D. ULLMAN [1972]. "Flow graph reducibility," *SIAM J. Computing* **1**, 188-202.

HECHT, M. S. AND J. D. ULLMAN [1974]. "Characterizations of reducible flow graphs," *J. ACM* **21**, 367-375.

HECHT, M. S. AND J. D. ULLMAN [1975]. "A simple algorithm for global data flow analysis programs," *SIAM J. Computing* **4**, 519-532.

HEIJENOORT, J. VAN [1967]. *From Frege to Gödel*, Harvard Univ. Press, Cambridge, Mass.

HENNESSY, J. [1981]. "Program optimization and exception handling," *Eighth Annual ACM Symposium on Principles of Programming Languages*, 200-206.

HENNESSY, J. [1982]. "Symbolic debugging of optimized code," *TOPLAS* **4:3**, 323-344.

HENRY, R. R. [1984]. *Graham-Glanville Code Generators*, Ph. D. Thesis, Univ. of California, Berkeley.

HEXT, J. B. [1967]. "Compile time type-matching," *Computer J.* **9**, 365-369.

HINDLEY, R. [1969]. "The principal type-scheme of an object in combinatory logic," *Trans. AMS* **146**, 29-60.

HOARE, C. A. R. [1962a]. "Quicksort," *Computer J.* **5:1**, 10-15.

HOARE, C. A. R. [1962b]. "Report on the Elliott Algol translator," *Computer J.* **5:2**, 127-129.

HOFFMAN, C. M. AND M. J. O'DONNELL [1982]. "Pattern matching in trees," *J. ACM* **29:1**, 68-95.

HOPCROFT, J. E. AND R. M. KARP [1971]. "An algorithm for testing the equivalence of finite automata," TR-71-114, Dept. of Computer Science, Cornell Univ. See Aho, Hopcroft, and Ullman [1974], pp. 143-145.

HOPCROFT, J. E. AND J. D. ULLMAN [1969]. *Formal Languages and Their Relation to Automata*, Addison-Wesley, Reading, Mass.

HOPCROFT, J. E. AND J. D. ULLMAN [1973]. "Set merging algorithms," *SIAM J. Computing* **2:3**, 294-303.

HOPCROFT, J. E. AND J. D. ULLMAN [1979]. *Introduction to Automata Theory, Languages, and Computation*, Addison-Wesley, Reading, Mass.

HORNING, J. J. [1976]. "What the compiler should tell the user," in Bauer and Eickel [1976].

HORWITZ, L. P., R. M. KARP, R. E. MILLER, AND S. WINOGRAD [1966]. "Index register allocation," *J. ACM* **13:1**, 43-61.

HUET, G. AND G. KAHN (EDS.) [1975]. *Proving and Improving Programs*, Colloque IRIA, Arc-et-Senans, France.

HUET, G. AND J.-J. LEVY [1979]. "Call-by-need computations in nonambiguous linear term rewriting systems," Rapport de Recherche 359, INRIA Laboria, Rocquencourt.

HUFFMAN, D. A. [1954]. "The synthesis of sequential machines," *J. Franklin Inst.* **257**, 3-4, 161, 190, 275-303.

HUNT, J. W. AND M. D. MCILROY [1976]. "An algorithm for differential file comparison," Computing Science Technical Report 41, AT&T Bell Laboratories, Murray Hill, N. J.

HUNT, J. W. AND T. G. SZYMANSKI [1977]. "A fast algorithm for computing longest common subsequences," *Comm. ACM* **20:5**, 350-353.

HUSKEY, H. D., M. H. HALSTEAD, AND R. MCARTHUR [1960]. "Neliac — a dialect of Algol," *Comm. ACM* **3:8**, 463-468.

ICHBIAH, J. D. AND S. P. MORSE [1970]. "A technique for generating almost optimal Floyd-Evans productions for precedence grammars," *Comm. ACM* **13:8**, 501-508.

INGALLS, D. H. H. [1978]. "The Smalltalk-76 programming system design and implementation," *Fifth Annual ACM Symposium on Principles of Programming Languages*, 9-16.

INGERMAN, P. Z. [1967]. "Panini-Backus form suggested," *Comm. ACM* **10:3**, 137.

IRONS, E. T. [1961]. "A syntax directed compiler for Algol 60," *Comm. ACM* **4:1**, 51-55.

IRONS, E. T. [1963]. "An error correcting parse algorithm," *Comm. ACM* **6:11**, 669-673.

IVERSON, K. [1962]. *A Programming Language*, Wiley, New York.

JANAS, J. M. [1980]. "A comment on "Operator identification in Ada" by Ganzinger and Ripken," *ACM SIGPLAN Notices* **15:9** (September) 39-43.

JARVIS, J. F. [1976]. "Feature recognition in line drawings using regular expressions," *Proc. 3rd Intl. Joint Conf. on Pattern Recognition*, 189-192.

JAZAYERI, M., W. F. OGDEN, AND W. C. ROUNDS [1975]. "The intrinsic exponential complexity of the circularity problem for attribute grammars," *Comm. ACM* **18:12**, 697-706.

JAZAYERI, M. AND D. POZEFSKY [1981]. "Space-efficient storage management in an attribute grammar evaluator," *TOPLAS* **3:4**, 388-404.

JAZAYERI, M. AND K. G. WALTER [1975]. "Alternating semantic evaluator," *Proc. ACM Annual Conference*, 230-234.

JENSEN, K. AND N. WIRTH [1975]. *Pascal User Manual and Report*, Springer-

Verlag, New York.

JOHNSON, S. C. [1975]. "Yacc – yet another compiler compiler," Computing Science Technical Report 32, AT&T Bell Laboratories, Murray Hill, N. J.

JOHNSON, S. C. [1978]. "A portable compiler: theory and practice," *Fifth Annual ACM Symposium on Principles of Programming Languages*, 97-104.

JOHNSON, S. C. [1979]. "A tour through the portable C compiler," AT&T Bell Laboratories, Murray Hill, N. J.

JOHNSON, S. C. [1983]. "Code generation for silicon," *Tenth Annual ACM Symposium on Principles of Programming Languages*, 14-19.

JOHNSON, S. C. AND M. E. LESK [1978]. "Language development tools," *Bell System Technical J.* **57:6**, 2155-2175.

JOHNSON, S. C. AND D. M. RITCHIE [1981]. "The C language calling sequence," Computing Science Technical Report 102, AT&T Bell Laboratories, Murray Hill, N. J.

JOHNSON, W. L., J. H. PORTER, S. I. ACKLEY, AND D. T. ROSS [1968]. "Automatic generation of efficient lexical processors using finite state techniques," *Comm. ACM* **11:12**, 805-813.

JOHNSSON, R. K. [1975]. *An Approach to Global Register Allocation*, Ph. D. Thesis, Carnegie-Mellon Univ., Pittsburgh, Pa.

JOLIAT, M. L. [1976]. "A simple technique for partial elimination of unit productions from LR(k) parser tables," *IEEE Trans. on Computers* **C-25:7**, 763-764.

JONES, N. D. [1980]. *Semantics Directed Compiler Generation*, Lecture Notes in Computer Science **94**, Springer-Verlag, Berlin.

JONES, N. D. AND C. M. MADSEN [1980]. "Attribute-influenced LR parsing," in Jones [1980], pp. 393-407.

JONES, N. D. AND S. S. MUCHNICK [1976]. "Binding time optimization in programming languages," *Third ACM Symposium on Principles of Programming Languages*, 77-94.

JOURDAN, M. [1984]. "Strongly noncircular attribute grammars and their recursive evaluation," *ACM SIGPLAN Notices* **19:6**, 81-93.

KAHN, G., D. B. MACQUEEN, AND G. PLOTKIN [1984]. *Semantics of Data Types*, Lecture Notes in Computer Science **173**, Springer-Verlag, Berlin.

KAM, J. B. AND J. D. ULLMAN [1976]. "Global data flow analysis and iterative algorithms," *J. ACM* **23:1**, 158-171.

KAM, J. B. AND J. D. ULLMAN [1977]. "Monotone data flow analysis frameworks," *Acta Informatica* **7:3**, 305-318.

KAPLAN, M. AND J. D. ULLMAN [1980]. "A general scheme for the automatic inference of variable types," *J. ACM* **27:1**, 128-145.

KASAMI, T. [1965]. "An efficient recognition and syntax analysis algorithm for context-free languages," AFCRL-65-758, Air Force Cambridge Research Laboratory, Bedford, Mass.

KASAMI, T., W. W. PETERSON, AND N. TOKURA [1973]. "On the capabilities of while, repeat, and exit statements," *Comm. ACM* **16:8**, 503-512.

KASTENS, U. [1980]. "Ordered attribute grammars," *Acta Informatica* **13:3**, 229-256.

KASTENS, U., B. HUTT, AND E. ZIMMERMANN [1982]. *GAG: A Practical Compiler Generator*, Lecture Notes in Computer Science **141**, Springer-Verlag, Berlin.

KASYANOV, V. N. [1973]. "Some properties of fully reducible graphs," *Information Processing Letters* **2:4**, 113-117.

KATAYAMA, T. [1984]. "Translation of attribute grammars into procedures," *TOPLAS* **6:3**, 345-369.

KENNEDY, K. [1971]. "A global flow analysis algorithm," *Intern. J. Computer Math. Section A* **3**, 5-15.

KENNEDY, K. [1972]. "Index register allocation in straight line code and simple loops," in Rustin [1972], pp. 51-64.

KENNEDY, K. [1976]. "A comparison of two algorithms for global flow analysis," *SIAM J. Computing* **5:1**, 158-180.

KENNEDY, K. [1981]. "A survey of data flow analysis techniques," in Muchnick and Jones [1981], pp. 5-54.

KENNEDY, K. AND J. RAMANATHAN [1979]. "A deterministic attribute grammar evaluator based on dynamic sequencing," *TOPLAS* **1:1**, 142-160.

KENNEDY, K. AND S. K. WARREN [1976]. "Automatic generation of efficient evaluators for attribute grammars," *Third ACM Symposium on Principles of Programming Languages*, 32-49.

KERNIGHAN, B. W. [1975]. "Ratfor – a preprocessor for a rational Fortran," *Software—Practice and Experience* **5:4**, 395-406.

KERNIGHAN, B. W. [1982]. "PIC – a language for typesetting graphics," *Software—Practice and Experience* **12:1**, 1-21.

KERNIGHAN, B. W. AND L. L. CHERRY [1975]. "A system for typesetting mathematics," *Comm. ACM* **18:3**, 151-157.

KERNIGHAN, B. W. AND R. PIKE [1984]. *The UNIX Programming Environment*, Prentice-Hall, Englewood Cliffs, N. J.

KERNIGHAN, B. W. AND D. M. RITCHIE [1978]. *The C Programming Language*, Prentice-Hall, Englewood Cliffs, N. J.

KILDALL, G. [1973]. "A unified approach to global program optimization," *ACM Symposium on Principles of Programming Languages*, 194-206.

KLEENE, S. C. [1956]. "Representation of events in nerve nets," in Shannon and McCarthy [1956], pp. 3-40.

KNUTH, D. E. [1962]. "A history of writing compilers," *Computers and Automation* (December) 8-18. Reprinted in Pollack [1972], pp. 38-56.

KNUTH, D. E. [1964]. "Backus Normal Form vs. Backus Naur Form," *Comm. ACM* **7:12**, 735-736.

KNUTH, D. E. [1965]. "On the translation of languages from left to right," *Information and Control* **8:6**, 607-639.

KNUTH, D. E. [1968]. "Semantics of context-free languages," *Mathematical Systems Theory* **2:2**, 127-145. Errata **5:1** (1971) 95-96.

KNUTH, D. E. [1971a]. "Top-down syntax analysis," *Acta Informatica* **1:2**, 79-110.

KNUTH, D. E. [1971b]. "An empirical study of FORTRAN programs," *Software—Practice and Experience* **1:2**, 105-133.

KNUTH, D. E. [1973a]. *The Art of Computer Programming:* Vol. 1, 2nd. Ed., *Fundamental Algorithms*, Addison-Wesley, Reading, Mass.

KNUTH, D. E. [1973b]. *The Art of Computer Programming:* Vol. 3, *Sorting and Searching*, Addison-Wesley, Reading, Mass.

KNUTH, D. E. [1977]. "A generalization of Dijkstra's algorithm," *Information Processing Letters* **6**, 1-5.

KNUTH, D. E. [1984a]. *The TEXbook*, Addison-Wesley, Reading, Mass.

KNUTH, D. E. [1984b]. "Literate programming," *Computer J.* **28:2**, 97-111.

KNUTH, D. E. [1985,1986]. *Computers and Typesetting*, Vol. 1: *TEX*, Addison-Wesley, Reading, Mass. A preliminary version has been published under the title, "*TEX: The Program.*"

KNUTH, D. E., J. H. MORRIS, AND V. R. PRATT [1977]. "Fast pattern matching in strings," *SIAM J. Computing* **6:2**, 323-350.

KNUTH, D. E. AND L. TRABB PARDO [1977]. "Early development of programming languages," *Encyclopedia of Computer Science and Technology* **7**, Marcel Dekker, New York, 419-493.

KORENJAK, A. J. [1969]. "A practical method for constructing LR(k) processors," *Comm. ACM* **12:11**, 613-623.

KOSARAJU, S. R. [1974]. "Analysis of structured programs," *J. Computer and System Sciences* **9:3**, 232-255.

KOSKIMIES, K. AND K.-J. RAIHA [1983]. "Modelling of space-efficient one-pass translation using attribute grammars," *Software—Practice and Experience* **13**, 119-129.

KOSTER, C. H. A. [1971]. "Affix grammars," in Peck [1971], pp. 95-109.

KOU, L. [1977]. "On live-dead analysis for global data flow problems," *J. ACM* **24:3**, 473-483.

KRISTENSEN, B. B. AND O. L. MADSEN [1981]. "Methods for computing LALR(k) lookahead," *TOPLAS* **3:1**, 60-82.

KRON, H. [1975]. *Tree Templates and Subtree Transformational Grammars*, Ph. D. Thesis, Univ. of California, Santa Cruz.

LALONDE, W. R. [1971]. "An efficient LALR parser generator," Tech. Rep. 2, Computer Systems Research Group, Univ. of Toronto.

LALONDE, W. R. [1976]. "On directly constructing LR(k) parsers without chain reductions," *Third ACM Symposium on Principles of Programming Languages*, 127-133.

LALONDE, W. R., E. S. LEE, AND J. J. HORNING [1971]. "An LALR(k) parser generator," *Proc. IFIP Congress 71* **TA-3**, North-Holland, Amsterdam, 153-157.

LAMB, D. A. [1981]. "Construction of a peephole optimizer," *Software—Practice and Experience* **11**, 638-647.

LAMPSON, B. W. [1982]. "Fast procedure calls," *ACM SIGPLAN Notices* **17:4** (April) 66-76.

LANDIN, P. J. [1964]. "The mechanical evaluation of expressions," *Computer J.* **6:4**, 308-320.

LECARME, O. AND M.-C. PEYROLLE-THOMAS [1978]. "Self-compiling compilers: an appraisal of their implementation and portability," *Software—Practice and Experience* **8**, 149-170.

LEDGARD, H. F. [1971]. "Ten mini-languages: a study of topical issues in programming languages," *Computing Surveys* **3:3**, 115-146.

LEINIUS, R. P. [1970]. *Error Detection and Recovery for Syntax Directed Compiler Systems*, Ph. D. Thesis, University of Wisconsin, Madison.

LENGAUER, T. AND R. E. TARJAN [1979]. "A fast algorithm for finding dominators in a flowgraph," *TOPLAS* **1**, 121-141.

LESK, M. E. [1975]. "Lex – a lexical analyzer generator," Computing Science Technical Report 39, AT&T Bell Laboratories, Murray Hill, N. J.

LEVERETT, B. W. [1982]. "Topics in code generation and register allocation," CMU CS-82-130, Computer Science Dept., Carnegie-Mellon Univ., Pittsburgh, Pennsylvania.

LEVERETT, B. W., R. G. G. CATTELL, S. O. HOBBS, J. M. NEWCOMER, A. H. REINER, B. R. SCHATZ, AND W. A. WULF [1980]. "An overview of the production-quality compiler-compiler project," *Computer* **13:8**, 38-40.

LEVERETT, B. W. AND T. G. SZYMANSKI [1980]. "Chaining span-dependent jump instructions," *TOPLAS* **2:3**, 274-289.

LEVY, J. P. [1975]. "Automatic correction of syntax errors in programming languages," *Acta Informatica* **4**, 271-292.

LEWIS, P. M., II, D. J. ROSENKRANTZ, AND R. E. STEARNS [1974]. "Attributed translations," *J. Computer and System Sciences* **9:3**, 279-307.

LEWIS, P. M., II, D. J. ROSENKRANTZ, AND R. E. STEARNS [1976]. *Compiler Design Theory*, Addison-Wesley, Reading, Mass.

LEWIS, P. M., II AND R. E. STEARNS [1968]. "Syntax-directed transduction," *J. ACM* **15:3**, 465-488.

LORHO, B. [1977]. "Semantic attribute processing in the system Delta," in Ershov and Koster [1977], pp. 21-40.

LORHO, B. [1984]. *Methods and Tools for Compiler Construction*, Cambridge Univ. Press.

LORHO, B. AND C. PAIR [1975]. "Algorithms for checking consistency of attribute grammars," in Huet and Kahn [1975], pp. 29-54.

LOW, J. AND P. ROVNER [1976]. "Techniques for the automatic selection of data structures," *Third ACM Symposium on Principles of Programming Languages*, 58-67.

LOWRY, E. S. AND C. W. MEDLOCK [1969]. "Object code optimization," *Comm. ACM* **12**, 13-22.

LUCAS, P. [1961]. "The structure of formula translators," *Elektronische Rechenanlagen* **3**, 159-166.

LUNDE, A. [1977]. "Empirical evaluation of some features of instruction set processor architectures," *Comm. ACM* **20:3**, 143-153.

LUNELL, H. [1983]. *Code Generator Writing Systems*, Ph. D. Thesis, Linköping University, Linköping, Sweden.

MACQUEEN, D. B., G. P. PLOTKIN, AND R. SETHI [1984]. "An ideal model of recursive polymorphic types," *Eleventh Annual ACM Symposium on Principles of Programming Languages*, 165-174.

MADSEN, O. L. [1980]. "On defining semantics by means of extended

attribute grammars," in Jones [1980], pp. 259-299.

MARILL, T. [1962]. "Computational chains and the simplification of computer programs," *IRE Trans. Electronic Computers* **EC-11:2**, 173-180.

MARTELLI, A. AND U. MONTANARI [1982]. "An efficient unification algorithm," *TOPLAS* **4:2**, 258-282.

MAUNEY, J. AND C. N. FISCHER [1982]. "A forward move algorithm for LL and LR parsers," *ACM SIGPLAN Notices* **17:4**, 79-87.

MAYOH, B. H. [1981]. "Attribute grammars and mathematical semantics," *SIAM J. Computing* **10:3**, 503-518.

MCCARTHY, J. [1963]. "Towards a mathematical science of computation," *Information Processing 1962*, North-Holland, Amsterdam, 21-28.

MCCARTHY, J. [1981]. "History of Lisp," in Wexelblat [1981], pp. 173-185.

MCCLURE, R. M. [1965]. "TMG – a syntax-directed compiler," *Proc. 20th ACM National Conf.*, 262-274.

MCCRACKEN, N. J. [1979]. *An Investigation of a Programming Language with a Polymorphic Type Structure*, Ph. D. Thesis, Syracuse University, Syracuse, N. Y.

MCCULLOUGH, W. S. AND W. PITTS [1943]. "A logical calculus of the ideas immanent in nervous activity," *Bulletin of Math. Biophysics* **5**, 115-133.

MCKEEMAN, W. M. [1965]. "Peephole optimization," *Comm. ACM* **8:7**, 443-444.

MCKEEMAN, W. M. [1976]. "Symbol table access," in Bauer and Eickel [1976], pp. 253-301.

MCKEEMAN, W. M., J. J. HORNING, AND D. B. WORTMAN [1970]. *A Compiler Generator*, Prentice-Hall, Englewood Cliffs, N. J.

MCNAUGHTON, R. AND H. YAMADA [1960]. "Regular expressions and state graphs for automata," *IRE Trans. on Electronic Computers* **EC-9:1**, 38-47.

MEERTENS, L. [1983]. "Incremental polymorphic type checking in B," *Tenth ACM Symposium on Principles of Programming Languages*, 265-275.

METCALF, M. [1982]. *Fortran Optimization*, Academic Press, New York.

MILLER, R. E. AND J. W. THATCHER (EDS.) [1972]. *Complexity of Computer Computations*, Academic Press, New York.

MILNER, R. [1978]. "A theory of type polymorphism in programming," *J. Computer and System Sciences* **17:3**, 348-375.

MILNER, R. [1984]. "A proposal for standard ML," *ACM Symposium on Lisp and Functional Programming*, 184-197.

MINKER, J. AND R. G. MINKER [1980]. "Optimization of boolean expressions – historical developments," *A. of the History of Computing* **2:3**, 227-238.

MITCHELL, J. C. [1984]. "Coercion and type inference," *Eleventh ACM Symposium on Principles of Programming Languages*, 175-185.

MOORE, E. F. [1956]. "Gedanken experiments in sequential machines," in Shannon and McCarthy [1956], pp. 129-153.

MOREL, E. AND C. RENVOISE [1979]. "Global optimization by suppression of partial redundancies," *Comm. ACM* **22**, 96-103.

MORRIS, J. H. [1968a]. *Lambda-Calculus Models of Programming Languages*, Ph. D. Thesis, MIT, Cambridge, Mass.

MORRIS, R. [1968b]. "Scatter storage techniques," *Comm. ACM* **11:1**, 38-43.

MOSES, J. [1970]. "The function of FUNCTION in Lisp," *SIGSAM Bulletin* **15** (July) 13-27.

MOULTON, P. G. AND M. E. MULLER [1967]. "DITRAN – a compiler emphasizing diagnostics," *Comm. ACM* **10:1**, 52-54.

MUCHNICK, S. S. AND N. D. JONES [1981]. *Program Flow Analysis: Theory and Applications*, Prentice-Hall, Englewood Cliffs, N. J.

NAKATA, I. [1967]. "On compiling algorithms for arithmetic expressions," *Comm. ACM* **10:8**, 492-494.

NAUR, P. (ED.) [1963]. "Revised report on the algorithmic language Algol 60," *Comm. ACM* **6:1**, 1-17.

NAUR, P. [1965]. "Checking of operand types in Algol compilers," *BIT* **5**, 151-163.

NAUR, P. [1981]. "The European side of the last phase of the development of Algol 60," in Wexelblat [1981], pp. 92-139, 147-161.

NEWEY, M. C., P. C. POOLE, AND W. M. WAITE [1972]. "Abstract machine modelling to produce portable software – a review and evaluation," *Software—Practice and Experience* **2:2**, 107-136.

NEWEY, M. C. AND W. M. WAITE [1985]. "The robust implementation of sequence-controlled iteration," *Software—Practice and Experience* **15:7**, 655-668.

NICHOLLS, J. E. [1975]. *The Structure and Design of Programming Languages*, Addison-Wesley, Reading, Mass.

NIEVERGELT, J. [1965]. "On the automatic simplification of computer code," *Comm. ACM* **8:6**, 366-370.

NORI, K. V., U. AMMANN, K. JENSEN, H. H. NAGELI, AND CH. JACOBI [1981]. "Pascal P implementation notes," in Barron [1981], pp. 125-170.

OSTERWEIL, L. J. [1981]. "Using data flow tools in software engineering," in Muchnick and Jones [1981], pp. 237-263.

PAGER, D. [1977a]. "A practical general method for constructing LR(k) parsers," *Acta Informatica* **7**, 249-268.

PAGER, D. [1977b]. "Eliminating unit productions from LR(k) parsers," *Acta Informatica* **9**, 31-59.

PAI, A. B. AND R. B. KIEBURTZ [1980]. "Global context recovery: a new strategy for syntactic error recovery by table-driven parsers," *TOPLAS* **2:1**, 18-41.

PAIGE, R. AND J. T. SCHWARTZ [1977]. "Expression continuity and the formal differentiation of algorithms," *Fourth ACM Symposium on Principles of Programming Languages*, 58-71.

PALM, R. C., JR. [1975]. "A portable optimizer for the language C," M. Sc. Thesis, MIT, Cambridge, Mass.

PARK, J. C. H., K. M. CHOE, AND C. H. CHANG [1985]. "A new analysis of LALR formalisms," *TOPLAS* **7:1**, 159-175.

PATERSON, M. S. AND M. WEGMAN [1978]. "Linear unification," *J. Computer and System Sciences* **16:2**, 158-167.

PECK, J. E. L. [1971]. *Algol 68 Implementation*, North-Holland, Amsterdam.

PENNELLO, T. AND F. DEREMER [1978]. "A forward move algorithm for LR error recovery," *Fifth Annual ACM Symposium on Principles of Programming Languages*, 241-254.

PENNELLO, T., F. DEREMER, AND R. MEYERS [1980]. "A simplified operator identification scheme for Ada," *ACM SIGPLAN Notices* **15:7** (July-August) 82-87.

PERSCH, G., G. WINTERSTEIN, M. DAUSSMANN, AND S. DROSSOPOULOU [1980]. "Overloading in preliminary Ada," *ACM SIGPLAN Notices* **15:11** (November) 47-56.

PETERSON, W. W. [1957]. "Addressing for random access storage," *IBM J. Research and Development* **1:2**, 130-146.

POLLACK, B. W. [1972]. *Compiler Techniques*, Auerbach Publishers, Princeton, N. J.

POLLOCK, L. L. AND M. L. SOFFA [1985]. "Incremental compilation of locally optimized code," *Twelfth Annual ACM Symposium on Principles of Programming Languages*, 152-164.

POWELL, M. L. [1984]. "A portable optimizing compiler for Modula-2," *ACM SIGPLAN Notices* **19:6**, 310-318.

PRATT, T. W. [1984]. *Programming Languages: Design and Implementation*, 2nd Ed., Prentice-Hall, Englewood Cliffs, N. J.

PRATT, V. R. [1973]. "Top down operator precedence," *ACM Symposium on Principles of Programming Languages*, 41-51.

PRICE, C. E. [1971]. "Table lookup techniques," *Computing Surveys* **3:2**, 49-65.

PROSSER, R. T. [1959]. "Applications of boolean matrices to the analysis of flow diagrams," *AFIPS Eastern Joint Computer Conf.*, Spartan Books, Baltimore, Md., 133-138.

PURDOM, P. AND C. A. BROWN [1980]. "Semantic routines and LR(k) parsers," *Acta Informatica* **14:4**, 299-315.

PURDOM, P. W. AND E. F. MOORE [1972]. "Immediate predominators in a directed graph," *Comm. ACM* **15:8**, 777-778.

RABIN, M. O. AND D. SCOTT. [1959]. "Finite automata and their decision problems," *IBM J. Research and Development* **3:2**, 114-125.

RADIN, G. AND H. P. ROGOWAY [1965]. "NPL: Highlights of a new programming language," *Comm. ACM* **8:1**, 9-17.

RAIHA, K.-J. [1981]. *A Space Management Technique for Multi-Pass Attribute Evaluators*, Ph. D. Thesis, Report A-1981-4, Dept. of Computer Science, University of Helsinki.

RAIHA, K.-J. AND M. SAARINEN [1982]. "Testing attribute grammars for circularity," *Acta Informatica* **17**, 185-192.

RAIHA, K.-J., M. SAARINEN, M. SARJAKOSKI, S. SIPPU, E. SOISALON-SOININEN, AND M. TIENARI [1983]. "Revised report on the compiler writing system HLP78," Report A-1983-1, Dept. of Computer Science, University of Helsinki.

RANDELL, B. AND L. J. RUSSELL [1964]. *Algol 60 Implementation*, Academic Press, New York.

REDZIEJOWSKI, R. R. [1969]. "On arithmetic expressions and trees," *Comm. ACM* **12:2**, 81-84.

REIF, J. H. AND H. R. LEWIS [1977]. "Symbolic evaluation and the global value graph," *Fourth ACM Symposium on Principles of Programming Languages*, 104-118.

REISS, S. P. [1983]. "Generation of compiler symbol processing mechanisms from specifications," *TOPLAS* **5:2**, 127-163.

REPS, T. W. [1984]. *Generating Language-Based Environments*, MIT Press, Cambridge, Mass.

REYNOLDS, J. C. [1985]. "Three approaches to type structure," *Mathematical Foundations of Software Development*, Lecture Notes in Computer Science **185**, Springer-Verlag, Berlin, 97-138.

RICHARDS, M. [1971]. "The portability of the BCPL compiler," *Software—Practice and Experience* **1:2**, 135-146.

RICHARDS, M. [1977]. "The implementation of the BCPL compiler," in P. J. Brown (ed.), *Software Portability: An Advanced Course*, Cambridge University Press.

RIPKEN, K. [1977]. "Formale beschreibun von maschinen, implementierungen und optimierender maschinen-codeerzeugung aus attributierten programmgraphe," TUM-INFO-7731, Institut für Informatik, Universität München, Munich.

RIPLEY, G. D. AND F. C. DRUSEIKIS [1978]. "A statistical analysis of syntax errors," *Computer Languages* **3**, 227-240.

RITCHIE, D. M. [1979]. "A tour through the UNIX C compiler," AT&T Bell Laboratories, Murray Hill, N. J.

RITCHIE, D. M. AND K. THOMPSON [1974]. "The UNIX time-sharing system," *Comm. ACM* **17:7**, 365-375.

ROBERTSON, E. L. [1979]. "Code generation and storage allocation for machines with span-dependent instructions," *TOPLAS* **1:1**, 71-83.

ROBINSON, J. A. [1965]. "A machine-oriented logic based on the resolution principle," *J. ACM* **12:1**, 23-41.

ROHL, J. S. [1975]. *An Introduction to Compiler Writing*, American Elsevier, New York.

ROHRICH, J. [1980]. "Methods for the automatic construction of error correcting parsers," *Acta Informatica* **13:2**, 115-139.

ROSEN, B. K. [1977]. "High-level data flow analysis," *Comm. ACM* **20**, 712-724.

ROSEN, B. K. [1980]. "Monoids for rapid data flow analysis," *SIAM J. Computing* **9:1**, 159-196.

ROSEN, S. [1967]. *Programming Systems and Languages*, McGraw-Hill, New York.

ROSENKRANTZ, D. J. AND R. E. STEARNS [1970]. "Properties of deterministic top-down grammars," *Information and Control* **17:3**, 226-256.

ROSLER, L. [1984]. "The evolution of C — past and future," *AT&T Bell Labs Technical Journal* **63:8**, 1685-1699.

RUSTIN, R. [1972]. *Design and Optimization of Compilers*, Prentice-Hall,

Englewood Cliffs, N.J.

RYDER, B. G. [1979]. "Constructing the call graph of a program," *IEEE Trans. Software Engineering* **SE-5:3**, 216-226.

RYDER, B. G. [1983]. "Incremental data flow analysis," *Tenth ACM Symposium on Principles of Programming Languages*, 167-176.

SAARINEN, M. [1978]. "On constructing efficient evaluators for attribute grammars," *Automata, Languages and Programming, Fifth Colloquium*, Lecture Notes in Computer Science **62**, Springer-Verlag, Berlin, 382-397.

SAMELSON, K. AND F. L. BAUER [1960]. "Sequential formula translation," *Comm. ACM* **3:2**, 76-83.

SANKOFF, D. AND J. B. KRUSKAL (EDS.) [1983]. *Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison*, Addison-Wesley, Reading, Mass.

SCARBOROUGH, R. G. AND H. G. KOLSKY [1980]. "Improved optimization of Fortran object programs," *IBM J. Research and Development* **24:6**, 660-676.

SCHAEFER, M. [1973]. *A Mathematical Theory of Global Program Optimization*, Prentice Hall, Englewood Cliffs, N. J.

SCHONBERG, E., J. T. SCHWARTZ, AND M. SHARIR [1981]. "An automatic technique for selection of data representations in SETL Programs," *TOPLAS* **3:2**, 126-143.

SCHORRE, D. V. [1964]. "Meta-II: a syntax-oriented compiler writing language," *Proc. 19th ACM National Conf.*, D1.3-1 - D1.3-11.

SCHWARTZ, J. T. [1973]. *On Programming: An Interim Report on the SETL Project*, Courant Inst., New York.

SCHWARTZ, J. T. [1975a]. "Automatic data structure choice in a language of very high level," *Comm. ACM* **18:12**, 722-728.

SCHWARTZ, J. T. [1975b]. "Optimization of very high level languages," *Computer Languages*. Part I: "Value transmission and its corollaries," **1:2**, 161-194; part II: "Deducing relationships of inclusion and membership," **1:3**, 197-218.

SEDGEWICK, R. [1978]. "Implementing Quicksort programs," *Comm. ACM* **21**, 847-857.

SETHI, R. [1975]. "Complete register allocation problems," *SIAM J. Computing* **4:3**, 226-248.

SETHI, R. AND J. D. ULLMAN [1970]. "The generation of optimal code for arithmetic expressions," *J. ACM* **17:4**, 715-728.

SHANNON, C. AND J. MCCARTHY [1956]. *Automata Studies*, Princeton University Press.

SHERIDAN, P. B. [1959]. "The arithmetic translator-compiler of the IBM Fortran automatic coding system," *Comm. ACM* **2:2**, 9-21.

SHIMASAKI, M., S. FUKAYA, K. IKEDA, AND T. KIYONO [1980]. "An analysis of Pascal programs in compiler writing," *Software—Practice and Experience* **10:2**, 149-157.

SHUSTEK, L. J. [1978]. "Analysis and performance of computer instruction sets," SLAC Report 205, Stanford Linear Accelerator Center, Stanford University, Stanford, California.

SIPPU, S. [1981]. "Syntax error handling in compilers," Rep. A-1981-1, Dept. of Computer Science, Univ. of Helsinki, Helsinki, Finland.

SIPPU, S. AND E. SOISALON-SOININEN [1983]. "A syntax-error-handling technique and its experimental analysis," *TOPLAS* **5:4**, 656-679.

SOISALON-SOININEN, E. [1980]. "On the space optimizing effect of eliminating single productions from LR parsers," *Acta Informatica* **14**, 157-174.

SOISALON-SOININEN, E. AND E. UKKONEN [1979]. "A method for transforming grammars into LL(k) form," *Acta Informatica* **12**, 339-369.

SPILLMAN, T. C. [1971]. "Exposing side effects in a PL/I optimizing compiler," *Information Processing 71*, North-Holland, Amsterdam, 376-381.

STEARNS, R. E. [1971]. "Deterministic top-down parsing," *Proc. 5th Annual Princeton Conf. on Information Sciences and Systems*, 182-188.

STEEL, T. B., JR. [1961]. "A first version of Uncol," *Western Joint Computer Conference*, 371-378.

STEELE, G. L., JR. [1984]. *Common LISP*, Digital Press, Burlington, Mass.

STOCKHAUSEN, P. F. [1973]. "Adapting optimal code generation for arithmetic expressions to the instruction sets available on present-day computers," *Comm. ACM* **16:6**, 353-354. Errata: **17:10** (1974) 591.

STONEBRAKER, M., E. WONG, P. KREPS, AND G. HELD [1976]. "The design and implementation of INGRES," *ACM Trans. Database Systems* **1:3**, 189-222.

STRONG, J., J. WEGSTEIN, A. TRITTER, J. OLSZTYN, O. MOCK, AND T. STEEL [1958]. "The problem of programming communication with changing machines: a proposed solution," *Comm. ACM* **1:8** (August) 12-18. Part 2: **1:9** (September) 9-15. Report of the Share Ad-Hoc committee on Universal Languages.

STROUSTRUP, B. [1986]. *The C++ Programming Language*, Addison-Wesley, Reading, Mass.

SUZUKI, N. [1981]. "Inferring types in Smalltalk," *Eighth ACM Symposium on Principles of Programming Languages*, 187-199.

SUZUKI, N. AND K. ISHIHATA [1977]. "Implementation of array bound checker," *Fourth ACM Symposium on Principles of Programming Languages*, 132-143.

SZYMANSKI, T. G. [1978]. "Assembling code for machines with span-dependent instructions," *Comm. ACM* **21:4**, 300-308.

TAI, K. C. [1978]. "Syntactic error correction in programming languages," *IEEE Trans. Software Engineering* **SE-4:5**, 414-425.

TANENBAUM, A. S., H. VAN STAVEREN, E. G. KEIZER, AND J. W. STEVENSON [1983]. "A practical tool kit for making portable compilers," *Comm. ACM* **26:9**, 654-660.

TANENBAUM, A. S., H. VAN STAVEREN, AND J. W. STEVENSON [1982]. "Using peephole optimization on intermediate code," *TOPLAS* **4:1**, 21-36.

TANTZEN, R. G. [1963]. "Algorithm 199: Conversions between calendar date and Julian day number," *Comm. ACM* **6:8**, 443.

TARHIO, J. [1982]. "Attribute evaluation during LR parsing," Report A-1982-4, Dept. of Computer Science, University of Helsinki.

TARJAN, R. E. [1974a]. "Finding dominators in directed graphs," *SIAM J. Computing* **3:1**, 62-89.

TARJAN, R. E. [1974b]. "Testing flow graph reducibility," *J. Computer and System Sciences* **9:3**, 355-365.

TARJAN, R. E. [1975]. "Efficiency of a good but not linear set union algorithm," *JACM* **22:2**, 215-225.

TARJAN, R. E. [1981]. "A unified approach to path problems," *J. ACM* **28:3**, 577-593. And "Fast algorithms for solving path problems," *J. ACM* **28:3** 594-614.

TARJAN, R. E. AND A. C. YAO [1979]. "Storing a sparse table," *Comm. ACM* **22:11**, 606-611.

TENNENBAUM, A. M. [1974]. "Type determination in very high level languages," NSO-3, Courant Institute of Math. Sciences, New York Univ.

TENNENT, R. D. [1981]. *Principles of Programming Languages*, Prentice-Hall International, Englewood Cliffs, N. J.

THOMPSON, K. [1968]. "Regular expression search algorithm," *Comm. ACM* **11:6**, 419-422.

TJIANG, S. W. K. [1986]. "Twig language manual," Computing Science

Technical Report 120, AT&T Bell Laboratories, Murray Hill, N. J.

TOKUDA, T. [1981]. "Eliminating unit reductions from LR(k) parsers using minimum contexts," *Acta Informatica* **15**, 447-470.

TRICKEY, H. W. [1985]. *Compiling Pascal Programs into Silicon*, Ph. D. Thesis, Stanford Univ.

ULLMAN, J. D. [1973]. "Fast algorithms for the elimination of common subexpressions," *Acta Informatica* **2**, 191-213.

ULLMAN, J. D. [1982]. *Principles of Database Systems*, 2nd Ed., Computer Science Press, Rockville, Md.

ULLMAN, J. D. [1984]. *Computational Aspects of VLSI*, Computer Science Press, Rockville, Md.

VYSSOTSKY, V. AND P. WEGNER [1963]. "A graph theoretical Fortran source language analyzer," manuscript, AT&T Bell Laboratories, Murray Hill, N. J.

WAGNER, R. A. [1974]. "Order-n correction for regular languages," *Comm. ACM* **16:5**, 265-268.

WAGNER, R. A. AND M. J. FISCHER [1974]. "The string-to-string correction problem," *J. ACM* **21:1**, 168-174.

WAITE, W. M. [1976a]. "Code generation," in Bauer and Eickel [1976], 302-332.

WAITE, W. M. [1976b]. "Optimization," in Bauer and Eickel [1976], 549-602.

WAITE, W. M. AND L. R. CARTER [1985]. "The cost of a generated parser," *Software—Practice and Experience* **15:3**, 221-237.

WASILEW, S. G. [1971]. *A Compiler Writing System with Optimization Capabilities for Complex Order Structures*, Ph. D. Thesis, Northwestern Univ., Evanston, Ill.

WATT, D. A. [1977]. "The parsing problem for affix grammars," *Acta Informatica* **8**, 1-20.

WEGBREIT, B. [1974]. "The treatment of data types in EL1," *Comm. ACM* **17:5**, 251-264.

WEGBREIT, B. [1975]. "Property extraction in well-founded property sets," *IEEE Trans. on Software Engineering* **1:3**, 270-285.

WEGMAN, M. N. [1983]. "Summarizing graphs by regular expressions," *Tenth Annual ACM Symposium on Principles of Programming Languages*, 203-216.

WEGMAN, M. N. AND F. K. ZADECK [1985]. "Constant propagation with conditional branches," *Twelfth Annual ACM Symposium on Principles of*

Programming Languages, 291-299.

WEGSTEIN, J. H. [1981]. "Notes on Algol 60," in Wexelblat [1981], pp. 126-127.

WEIHL, W. E. [1980]. "Interprocedural data flow analysis in the presence of pointers, procedure variables, and label variables," *Seventh Annual ACM Symposium on Principles of Programming Languages*, 83-94.

WEINGART, S. W. [1973]. *An Efficient and Systematic Method of Code Generation*, Ph. D. Thesis, Yale University, New Haven, Connecticut.

WELSH, J., W. J. SNEERINGER, AND C. A. R. HOARE [1977]. "Ambiguities and insecurities in Pascal," *Software—Practice and Experience* **7:6**, 685-696.

WEXELBLAT, R. L. [1981]. *History of Programming Languages*, Academic Press, New York.

WIRTH, N. [1968]. "PL 360 – a programming language for the 360 computers," *J. ACM* **15:1**, 37-74.

WIRTH, N. [1971]. "The design of a Pascal compiler," *Software—Practice and Experience* **1:4**, 309-333.

WIRTH, N. [1981]. "Pascal-S: A subset and its implementation," in Barron [1981], pp. 199-259.

WIRTH, N. AND H. WEBER [1966]. "Euler: a generalization of Algol and its formal definition: Part I," *Comm. ACM* **9:1**, 13-23.

WOOD, D. [1969]. "The theory of left factored languages," *Computer J.* **12:4**, 349-356.

WULF, W. A., R. K. JOHNSSON, C. B. WEINSTOCK, S. O. HOBBS, AND C. M. GESCHKE [1975]. *The Design of an Optimizing Compiler*, American Elsevier, New York.

YANNAKAKIS, M. [1985]. private communication.

YOUNGER, D. H. [1967]. "Recognition and parsing of context-free languages in time n^3," *Information and Control* **10:2**, 189-208.

ZELKOWITZ, M. V. AND W. G. BAIL [1974]. "Optimization of structured programs," *Software—Practice and Experience* **4:1**, 51-57.

Index

C

G

H

I

J

M

N

O

P